기민수

보안, 인프라, AI 보안 엔지니어

미디어솔루션에서 1년 11개월 동안 병역특례 산업기능요원으로 근무하며 대형병원, 대형 컨벤션센터, 5성급 호텔 등 4개 기업 고객의 키오스크 CMS와 운영 서버를 개발하고 운영했습니다. 고객 시스템에서 일어난 침해사고 두 건을 현장에서 대응했고, 웹 애플리케이션을 AWS에 다시 구축하며 외부에서 접근할 수 있는 경로를 줄였습니다. 지금은 개인 프로젝트로 AI 관련 보안 도구를 만들고 있습니다. LLM과 MCP 통신에서 개인정보와 기밀이 새어 나가지 않게 막는 프록시, 정적분석과 LLM 검증을 묶은 취약점 스캐너가 그 예입니다.

kiminsoo365@gmail.com · https://github.com/ki0915 · https://www.linkedin.com/in/minsoo-ki-388b2437b

사례 CASES

고객사 웹 서버 침해사고 두 건 대응과 AWS 재구축

보안 사례, 2023.04–2025.03

KISA 침해 통보와 서비스 중단 신고로 시작된 두 사고에 현장에서 대응하고, 무결성을 확인한 데이터만 AWS로 옮겨 서버를 다시 세웠습니다.

대형병원 키오스크 CMS의 권한 결함 수정과 보안점검 대응

보안 사례, 2023.04–2025.03

대형병원 키오스크를 원격으로 관리하는 CMS를 혼자 만들어 운영했고, 세션 검증보다 권한 부여가 먼저 일어나던 결함을 고쳐 병원 보안점검을 통과했습니다.

ETK-Scanner, 비용 상한이 있는 LLM 소스코드 취약점 진단

도구 사례, 2026.07

github.com/ki0915/etk-scanner

정적분석으로 의심 구간을 먼저 좁히고 LLM 검증은 필요한 곳에만 쓰는 취약점 스캐너를 만들어, 실습 백엔드와 오픈소스의 권한 문제를 찾았습니다.

Mcp-Guard, LLM과 MCP 통신의 개인정보 유출 방지 프록시

도구 사례, 2026.07

github.com/ki0915/Mcp-Guard

LLM과 MCP 도구로 나가는 요청에서 주민등록번호와 API 키 같은 민감 정보를 찾아 가리거나 막는 프록시를 만들었습니다.

EvidScope, AI 행동을 감사할 수 있게 근거를 묶는 파일럿

도구 사례, 2026.09–2026.10

github.com/ki0915/EvidScope

AI가 한 행동과 그때의 권한, 정책, 근거 문서, 사람의 판단을 서명 원장으로 이어 붙여 감사 담당자가 대조할 수 있게 한 로컬 파일럿입니다.

서버 7대 통합과 여러 겹의 백업 설계

보안 사례, 2023.04–2025.03

노후 서버 7대를 Hyper-V로 합치며 외부에서 바로 들어오던 원격 접속을 없애고, 스냅샷 하나에 기대던 복구를 여러 겹의 백업으로 바꿨습니다.

5성급 호텔 서버의 디스크 고갈 장애와 재발 방지

운영 사례, 2023.04–2025.03

배포 때마다 쌓인 Docker 이미지와 중지된 컨테이너가 디스크를 채워 서비스가 멈춘 장애를 복구하고, 주기 정리를 자동화해 같은 원인의 장애를 막았습니다.

프로젝트

PROJECTS

Docker 컨테이너 보안 가이드 저술

기술 문서, 2026년 개인 저술

github.com/ki0915/Docker-Security-Guide

위험도와 구현 비용을 기준으로 Docker 컨테이너 보안 대응을 다시 정리한 2026년 개인 기술 문서입니다.

TryHackMe 모의 침투 실습과 보고서

프로젝트

허가된 워게임 환경에서 모의 침투를 연습하고, 공격 경로와 개선 방안을 보고서로 정리한 학습 기록입니다.

WIKI

개인 학습, 2023년 개인 프로젝트

github.com/ki0915/wiki-backend

게시판과 권한별 접근 제한, 실시간 채팅을 갖춘 위키 서비스를 프론트엔드부터 백엔드까지 직접 만든 2023년 개인 프로젝트입니다.

Carrot 클론

개인 학습 프로젝트, 2022년

상품을 올리고 둘러보는 거래 서비스를 본떠 화면과 서버를 연결해 본 2022년 학습 프로젝트입니다.

Kiosk

학습용 개인 프로젝트, 2021.09–2022.01

2021년 9월부터 2022년 1월까지 주문 키오스크의 상품 선택부터 주문 전송까지 만들어 본 학습용 개인 프로젝트입니다.

보안 사례 • 2023.04–2025.03

고객사 웹 서버 침해사고 두 건 대응과 AWS 재구축

KISA 침해 통보와 서비스 중단 신고로 시작된 두 사고에 현장에서 대응하고, 무결성을 확인한 데이터만 AWS로 옮겨 서버를 다시 세웠습니다.

역할

현장 대응, 로그 분석, 복구와 AWS 재구축, 고객 보고

상황

미디어솔루션은 병원, 컨벤션센터, 호텔 같은 기업 고객의 키오스크 CMS와 운영 서버를 맡는 소기업이었습니다. 한 고객사의 웹 서버는 오래된 ASP.NET 위에서 돌고 있었고, 이 서버에서 침해사고가 두 번 일어났습니다. 두 사고 모두 제가 현장에 나가 대응했습니다.

어떻게 알았나

1차 사고는 KISA가 고객사 서버의 침해를 통보해 알게 됐습니다.

2차 사고는 서비스가 멈췄다는 고객 신고로 시작됐습니다. 서버의 CPU와 메모리가 비정상적으로 점유돼 있었고 공격자가 설치한 가상화 환경이 있어, 서버 자원을 몰래 쓰는 크립토마이닝을 의심했습니다.

무엇부터, 왜

무엇이 남아 있는지 보려면 시스템 접근과 관리자 계정부터 되찾아야 했습니다. 이 단계 없이는 로그도 프로세스도 확인할 수 없었기 때문입니다.

고객에게 조치가 필요한 이유를 설명할 때는 기존 .NET Framework 버전과 업데이트 버전의 공개 CVE 수, 사고 빈도를 근거로 삼았습니다. 이 자료로 AI를 활용해 위험 감소 추정치를 만들었고 출처를 함께 남겼습니다.

내가 한 것

- 현장에서 시스템 접근과 관리자 계정을 되찾고, IIS 로그를 따라가 응용프로그램 풀 안의 악성 실행을 루트킷으로 판단했습니다.
- 구형 미들웨어와 ASP.NET 관련 취약점을 점검하고 조치했습니다.
- MSSQL 데이터는 내부망의 격리된 PC로 옮겨 무결성을 확인한 뒤 분리했습니다.
- 2차 사고 복구 중 MSSQL 백업 복원이 멈춘 원인을 찾아, 업로드 검증 없이 저장된 과대 크기의 악성 이미지 파일을 분리하고 제거했습니다.
- 웹 애플리케이션을 AWS EC2에 새로 세우고 DB를 RDS로 옮겼습니다. 보안그룹은 운영 IP 허용목록만 열고 RDP 접근을 막았습니다.

함께 고친 과정

기술 담당자에게는 취약한 구성과 코드 문제를, 경영진에게는 서비스 중단과 복구 비용 관점의 권고를 따로 설명했습니다.

파트너사 담당자는 보안 조치에 드는 비용과 재발을 걱정했고, 서비스 복구 말고는 관심이 크지 않았습니다. 그래서 비용과 걸리는 시간을 그래프와 도형으로 보여 줬습니다. 설명을 들은 고객은 클라우드 이관과 주기적인 서버 관리에 합의했습니다.

확인과 재발 방지

파일과 입력 무결성 검증을 강화한 서버를 다시 배포했습니다. VPN을 쓸 수 없는 구간은 서비스 포트와 출발지 IP를 한정했습니다. AWS로 옮긴 뒤 제가 재직하는 동안 같은 원인의 공격은 다시 확인되지 않았습니다.

전과 후

항목	전	후
서버	고객사 Windows 서버와 오래된 ASP.NET	**AWS EC2와 RDS로 재구축**
관리 접근	–	**운영 IP 허용목록만 열고 RDP 차단**
업로드 파일	검증 없이 저장	**파일과 입력 무결성 검증 뒤 저장**
복구	–	**사고 인지부터 복구까지 4일**
같은 원인의 재발	–	**재직 중 확인되지 않음**

보안 사례 • 2023.04–2025.03

대형병원 키오스크 CMS의 권한 결함 수정과 보안점검 대응

대형병원 키오스크를 원격으로 관리하는 CMS를 혼자 만들어 운영했고, 세션 검증보다 권한 부여가 먼저 일어나던 결함을 고쳐 병원 보안점검을 통과했습니다.

역할

CMS 단독 설계, 개발, 운영과 보안점검 대응

상황

고객은 4만 평 규모의 대형병원이었습니다. 병원 내부망의 키오스크 약 200대는 장애가 날 때마다 담당자가 현장으로 가야 했습니다. 이 부담을 줄이려고 원격 관리 CMS를 혼자 만들어 운영했습니다. CMS에는 화면 미러링과 원격 명령, 예약 배포를 넣었습니다.

납품하는 CMS는 병원 보안점검을 통과해야 했고, 권한 분리와 계정 정책을 병원 기준에 맞춰야 했습니다.

어떻게 알았나

병원 보안 요구에 맞춰 인증과 권한 구조를 고치던 중에, Redis에서 세션 검증이 끝나기 전에 임시 권한이 먼저 붙는 구간을 찾았습니다. 요청 타이밍에 따라 검증되지 않은 세션이 권한을 쥘 수 있는 구조였습니다.

내가 한 것

- 권한 부여를 세션 검증 뒤로 옮겨 처리 순서를 바로잡았습니다.
- 서버 개인키로 서명하는 JWT 인증과 bcrypt 비밀번호 해시, 관리자와 운영자 역할을 나눈 RBAC를 구현했습니다.
- 90일마다 비밀번호를 바꾸게 하고, 로그인 실패 횟수 제한과 재사용 금지, 계정 잠금 정책을 적용했습니다.
- ORM으로 쿼리를 만들어 SQL을 직접 조립하는 곳을 줄였습니다.
- 연동 문제는 브라우저 개발자 도구의 Network 탭에서 엔드포인트와 요청 내용, Authorization 헤더의 JWT를 확인하며 분석했습니다.

확인과 재발 방지

납품한 CMS 전체가 병원 보안점검을 통과했습니다.

전과 후

항목	전	후
권한 부여 순서	요청, 임시 권한 부여, 세션 검증	**요청, 세션 검증, 권한 부여**
계정 정책	–	**정기 변경, 실패 횟수 제한, 재사용 금지, 계정 잠금**
키오스크 장애 처리	담당자가 현장으로 이동	**원격으로 풀 수 있는 장애는 CMS로 처리**

도구 사례 • 2026.07

ETK-Scanner, 비용 상한이 있는 LLM 소스코드 취약점 진단

정적분석으로 의심 구간을 먼저 좁히고 LLM 검증은 필요한 곳에만 쓰는 취약점 스캐너를 만들어, 실습 백엔드와 오픈소스의 권한 문제를 찾았습니다.

역할

파이프라인 설계, 구현, 검증

풀려던 문제

위험 함수 호출만 찾는 규칙 기반 도구는 권한 우회나 검증 우회처럼 위험 함수 없이 생기는 결함을 잡지 못합니다. 모든 함수를 LLM에 보내면 비용 때문에 CI에 붙이기 어렵습니다. 비용 대비 효율이 좋은 취약점 진단 도구를 만들고자 시작했습니다.

고른 설계와 이유

비용이 적은 단계부터 거르도록 정적분석, 저비용 모델 분류, 고비용 모델 검증 순서로 나눴습니다.

후보에는 eval 같은 위험 함수와 함께 validate나 auth처럼 보안 판단을 하는 함수를 넣었습니다. 권한 우회나 검증 우회는 위험 함수 호출 없이도 생기기 때문입니다.

직접 설계하고 만든 것

- Python 코드는 AST로, JavaScript와 TypeScript는 정규식으로 보안 판단 함수를 후보로 뽑습니다. 이 단계는 LLM 비용이 들지 않습니다.
- 저비용 모델이 후보마다 한 번 분류하고, 남은 소수만 고비용 모델이 여러 차례 문맥을 확인하며 검증합니다.
- 모든 LLM 호출의 비용을 기록하고, 정한 예산에 닿으면 분석을 멈춥니다.
- 결과를 SARIF로 내보내 GitHub 보안 탭에 올리고, GitHub Actions에서 PR과 push마다 정적 모드를 돌려 검사가 실패하면 job을 실패로 처리합니다.

함께 쓰고 고친 과정

이 스캐너를 활용해 한 오픈소스 프로젝트의 인가 처리 문제를 소스 수준에서 분석했습니다. 재현 자료와 원인 분석을 유지관리자에게 보냈고, 영향 확인과 수정 예정 회신을 받았습니다. 공개 전이라 프로젝트 이름과 세부 내용은 적지 않습니다.

검증 방법

직접 만든 TypeScript 실습 백엔드를 검사하자 소스에 박힌 JWT 서명 키가 잡혔습니다. 이 키를 알면 누구나 토큰을 위조할 수 있습니다. 키를 환경변수로 옮기기 전과 후를 실제 HTTP 서버로 비교했습니다.

공개 SecurityEval 데이터의 취약 코드에도 스캐너를 돌려 몇 개를 찾는지 확인했습니다.

적용 전과 후

항목	전	후
위조 토큰 요청	HTTP 200, 보호 데이터 노출	**HTTP 401로 차단**
SecurityEval 취약 코드	–	**121개 중 111개 탐지**

ETK-Scanner

역할	저장소
정적 후보 추출·LLM 단계 검증·비용 제어·결과 출력 파이프라인 개발	github.com/ki0915/etk-scanner

로컬 코드 확인 2026-10-16

기능 명세

기능	하는 일	확인
보안 판단 함수 탐지	Python 코드는 AST로, JavaScript와 TypeScript 코드는 정규식으로 읽어 보안 판단이 들어간 함수를 후보로 뽑습니다.	공개 SecurityEval 데이터의 취약 코드 121개에 스캐너를 실행해 111개를 탐지했습니다.
단계별 LLM 검증	저비용 모델이 후보마다 한 번 질의해 취약점 가설과 신뢰도를 매기고, 남은 소수 후보만 고비용 모델이 여러 차례 질의하며 검증합니다.	
비용 기록과 예산 상한	모든 LLM 호출의 토큰과 비용을 누적해 기록하고, 정한 예산에 닿으면 분석을 멈춥니다.	
SARIF 출력	탐지 결과의 CWE, 심각도, 파일과 행 위치를 SARIF 2.1.0으로 변환해 GitHub 보안 탭에 올릴 수 있게 합니다.	
CI 연동	GitHub Actions에서 PR과 push에는 기본 정적 모드를 돌리고 LLM 모드는 수동으로 실행하며, 검사가 실패하면 마지막 job을 실패로 처리합니다.	

시연

```
$ python -m unittest discover -s tests
.....................................
----------------------------------------------------------------------
Ran 45 tests in 2.152s

OK
```

python -m unittest discover -s tests 실행 결과

만든 방법

1
후보 추출
Python AST와 JS, TS 정규식으로 보안 판단 함수를 넓게 뽑으며, 이 단계에는 LLM 비용이 들지 않습니다.

2
저비용 분류
한 번의 질의로 후보마다 취약점 가설과 신뢰도를 매겨 검증할 대상을 줄입니다.

3
고비용 검증
남은 소수 후보만 여러 차례 질의하며 코드를 따라가 검증합니다.

4
필터와 중복 확인
기능 버그와 보안 취약점을 나누고, OSV 데이터베이스에서 이미 알려진 취약점인지 조회합니다.

5
결과 출력
남은 결과를 SARIF 2.1.0으로 내보내고, 모든 단계의 호출 비용을 합산해 기록합니다.

언어	Python
실행 환경	GitHub Actions
주요 라이브러리	bandit, anthropic, PyYAML

- **Claude Haiku와 Sonnet** 분류는 비용이 적은 Haiku에, 검증은 Sonnet에 나눠 맡겨 비싼 모델은 소수 후보에만 쓰도록 했습니다.

오픈소스 보안 연구와 책임 있는 제보

기능 명세

기능	하는 일	확인
인가 로직 분석	소스 코드에서 권한을 확인하는 처리를 따라가며 문제가 생기는 원인을 좁혔습니다.	
재현 자료와 원인 분석 전달	문제를 재현하는 자료와 원인 분석을 정리해 유지관리자에게 보냈습니다.	유지관리자가 영향을 확인하고 수정할 예정이라고 회신했습니다.

도구 사례 • 2026.07

Mcp-Guard, LLM과 MCP 통신의 개인정보 유출 방지 프록시

LLM과 MCP 도구로 나가는 요청에서 주민등록번호와 API 키 같은 민감 정보를 찾아 가리거나 막는 프록시를 만들었습니다.

역할

정책과 탐지 설계, 구현, 검증

풀려던 문제

생성형 AI를 업무에 쓰면 사람이 붙여 넣은 문서나 로그에 개인정보와 시크릿이 섞이기 쉽습니다. MCP 서버는 표준 입출력으로 JSON-RPC 메시지를 주고받는 경우가 많습니다. 그래서 네트워크 경계에 둔 HTTP 프록시만으로는 내용을 볼 수 없습니다.

문제를 확인한 방법

처음 버전은 원문 그대로만 검사했고, 띄어 쓴 주민등록번호나 전각 숫자, Base64로 감싼 값이 그대로 통과했습니다. 우회 기법을 넣은 합성 데이터로 그 차이를 쟀습니다.

고른 설계와 이유

HTTP 프록시와 stdio 어댑터에 같은 정책을 적용해 두 경로의 판단이 같게 했습니다. 검사는 원문 위치를 유지한 파생 문자열로 하고, 마스킹은 원래 입력의 해당 범위에 적용했습니다. 판단이 어려우면 막고, 감사 로그에는 원래 값을 남기지 않게 했습니다.

직접 설계하고 만든 것

- HTTP 리버스 프록시는 요청과 응답 본문, URL, SSE 이벤트를 검사하고, stdio 어댑터는 MCP의 줄 단위 JSON-RPC를 같은 정책으로 검사합니다.
- 주민등록번호는 날짜와 검증 숫자로, 카드번호는 IIN과 Luhn 검사로, 계좌번호는 앞뒤의 은행 문맥으로 확인합니다.
- 유니코드 NFKC 정규화, 숫자 사이 공백과 보이지 않는 문자 제거, 크기를 제한한 Base64 디코딩으로 우회 표현을 풀어 검사합니다.
- 정책 스키마 검증과 시크릿 분리, 만료 시간이 있는 허용 예외, Prometheus 지표, Dockerfile과 Helm 차트를 만들었습니다.

함께 쓰고 고친 과정

우회 사례와 테스트 초안은 Claude Code와 Codex의 제안을 참고했습니다. 탐지와 정책 기준, 정규화 방식, 구현 검토와 최종 반영은 직접 판단했습니다.

검증 방법

우회 기법을 넣은 합성 데이터로 정규화 전후의 탐지 수와 오탐을 비교했습니다. 합성 데이터는 민감 사례와 정상 사례로 나눴습니다. 프록시를 거칠 때 늘어나는 지연은 로컬에서 같은 요청을 짝지어 비교했습니다.

적용 전과 후

항목	전	후
민감 우회 사례 탐지	60건 중 10건	**60건 중 56건**
정상 사례 오탐	30건 중 0건	**30건 중 0건**

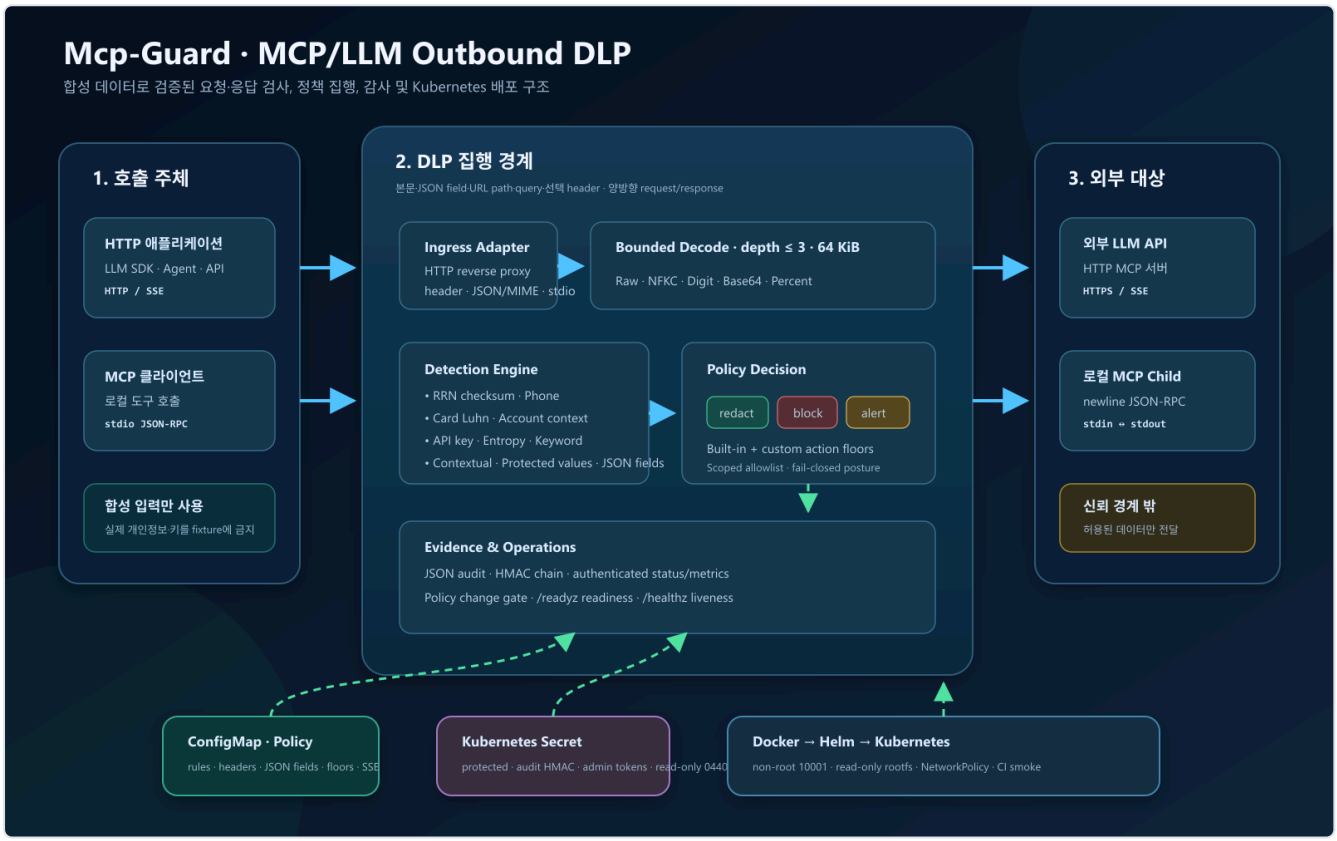
항목	전	후
HTTP 프록시 추가 지연	–	**평균 3.10ms**

Mcp-Guard

역할
정책·탐지 설계, 구현 및 AI 제안 검수

저장소
github.com/ki0915/Mcp-Guard

로컬 코드 확인 2026-10-16



Mcp-Guard 아키텍처

기능 명세

기능	하는 일	확인
HTTP 리버스 프록시	외부 LLM API로 가는 요청과 돌아오는 응답을 받아 경로, 쿼리, 본문과 SSE 스트림을 검사한 뒤 정책에 맞게 내보냅니다.	로컬 환경에서 1,102바이트 요청 200쌍을 비교해, 프록시를 거칠 때 평균 3.10ms가 더 걸리는 것을 측정했습니다.
stdio 어댑터	표준입출력으로 한 줄씩 오가는 MCP의 JSON-RPC 요청과 응답을 받아 HTTP 프록시와 같은 정책으로 검사합니다.	
개인정보 탐지	주민등록번호는 날짜와 검증 숫자로, 카드번호는 IIN과 Luhn 검사로, 계좌번호는 앞뒤의 은행 문맥으로 확인해 가리거나 막습니다.	
시크릿과 보호값 탐지	API 키는 알려진 접두어와 엔트로피로 찾고, 조직이 등록한 보호값은 원문이나 SHA-256 해시와 일치하면 막거나 가립니다.	

기능	하는 일	확인
우회 표기 정규화	유니코드 표기 차이와 숫자 사이의 공백, 보이지 않는 문자를 정리하고 Base64는 크기를 제한해 한 번만 풀어 검사합니다.	우회 기법을 넣은 합성 데이터로 정규화 전후의 탐지 수를 비교했습니다.
정책과 감사 기록	설정 스키마를 검증하고 시크릿을 설정과 분리하며, 기한이 있는 허용 예외와 원래 값을 뺀 감사 로그, Prometheus 지표를 제공합니다.	

시연

```
$ python -m pytest -q
..................................................................... [ 21%]
.....................s.............................................. [ 42%]
..................................................................... [ 64%]
..................................................................... [ 85%]
.....................................................                [100%]
334 passed, 1 skipped in 3.26s
```

python -m pytest -q 실행 결과

만든 방법

1
가로채기
HTTP 리버스 프록시와 stdio 어댑터가 외부로 나가는 요청과 돌아오는 응답을 중간에서 받습니다.

2
정규화
NFKC 정규화와 숫자 구분자 정리, 크기를 제한한 Base64 디코딩으로 검사할 문자열을 만듭니다.

3
탐지
규칙마다 형식과 검증 숫자, 주변 문맥을 확인하고, 정규화한 문자열에서 찾은 위치를 원문 범위로 되돌립니다.

4
정책 적용
규칙에 정한 대로 원문을 가리거나 전송을 막거나 경보를 남기며, 판단할 수 없으면 차단합니다.

5
기록
원래 값을 남기지 않는 감사 로그와 Prometheus 지표를 쌓습니다.

언어	Python
실행 환경	Docker, GitHub Actions
주요 라이브러리	fastapi, uvicorn, httpx, pyyaml, regex

도구 사례 • 2026.09–2026.10

EvidScope, AI 행동을 감사할 수 있게 근거를 묶는 파일럿

AI가 한 행동과 그때의 권한, 정책, 근거 문서, 사람의 판단을 서명 원장으로 이어 붙여 감사 담당자가 대조할 수 있게 한 로컬 파일럿입니다.

역할
문제 정의와 우선순위, 모델과 비용 조건 결정, 최종 검토 (설계와 구현은 AI와 협업)

풀려던 문제

기업이 업무에 AI를 쓰면 결과 보고는 받지만, AI가 어떤 도구를 어떤 권한으로 불렀고 어떤 근거를 봤는지는 확인하기 어렵습니다. 감사 담당자는 AI의 행동과 도구 결과, 당시 권한과 정책 버전, 원본 문서를 대조하고 판단을 남길 수 있어야 합니다. AI 활동이 보여야 AI 거버넌스를 시작할 수 있다고 보고 기획했습니다.

적용 전과 후

항목	전	후
지속 구간 접수	600건 중 84건	**600건 모두**
지속 구간 응답 p95	약 6초	**206.81ms**
감사자의 거버넌스 변경 요청	받아들여짐	**30건 모두 거부, 서명 상태 그대로**

자세한 내용 ki0915.github.io/Profile/docs/projects/evidscope.html

보안 사례 • 2023.04–2025.03

서버 7대 통합과 여러 겹의 백업 설계

노후 서버 7대를 Hyper-V로 합치며 외부에서 바로 들어오던 원격 접속을 없애고, 스냅샷 하나에 기대던 복구를 여러 겹의 백업으로 바꿨습니다.

역할

서버 점검, 가상화 재설계와 이관, 백업과 이중화 설계, 절차 문서화

상황

사내 서버 7대는 오래된 장비였습니다. 웹 서버와 DB에는 외부에서 바로 원격 접속할 수 있었고, 출결 서버는 Windows XP 기반이었습니다. 장애가 나면 기댈 수 있는 것은 Hyper-V 스냅샷 하나였고, 담당자가 없을 때도 복구할 수 있는 절차가 필요했습니다.

전과 후

항목	전	후
복구 수단	Hyper-V 스냅샷 하나	**변경 제한 백업, AWS 서버 이미지, MSSQL 복제**
외부 원격 접속	웹 서버와 DB에 바로 접속	**내부망 경로로만 접근**
API 장애	이중화 없음	**실패하면 이중화 서버로 전환**

자세한 내용 ki0915.github.io/Profile/docs/projects/backup-servers.html

운영 사례 • 2023.04–2025.03

5성급 호텔 서버의 디스크 고갈 장애와 재발 방지

배포 때마다 쌓인 Docker 이미지와 중지된 컨테이너가 디스크를 채워 서비스가 멈춘 장애를 복구하고, 주기 정리를 자동화해 같은 원인의 장애를 막았습니다.

역할

장애 원인 확인, 정리와 자동화, 운영 문서 작성

상황

5성급 호텔 고객사의 서버는 Ubuntu 위에서 Docker 컨테이너로 서비스를 돌렸습니다. 관리자 페이지와 DB, 서버 상태는 SSL VPN으로 접속해 점검했습니다. Grafana와 Prometheus 지표가 임계치에 가까워지면 미리 조치해 왔습니다.

전과 후

항목	전	후
디스크 정리	–	**cron으로 주기 실행**
재현 실험의 배포	14회 중 7회 실패	**14회 모두 성공**

자세한 내용 ki0915.github.io/Profile/docs/projects/docker-disk-outage.html

기술 문서, 2026년 개인 저술

Docker 컨테이너 보안 가이드 저술

위험도와 구현 비용을 기준으로 Docker 컨테이너 보안 대응을 다시 정리한 2026년 개인 기술 문서입니다.

역할
작성자

저장소
github.com/ki0915/Docker-Security-Guide

저장소 확인 2026-10-16

OWASP Docker Top 10, CIS Docker Benchmark, NIST SP 800-190을 참고해 D01부터 D10까지의 대응 방법을 위험도와 구현 비용에 따라 재구성했습니다. 항목마다 설정 코드 예와 설명을 붙였습니다.

자세한 내용 ki0915.github.io/Profile/docs/projects/docker-security-guide.html

프로젝트

TryHackMe 모의 침투 실습과 보고서

허가된 워게임 환경에서 모의 침투를 연습하고, 공격 경로와 개선 방안을 보고서로 정리한 학습 기록입니다.

TryHackMe의 Jr Penetration Tester 경로와 웹 해킹 모듈을 실습했습니다. 정찰부터 침투, 권한 상승, 개선 권고까지를 OSCP 보고서 양식으로 작성하며 보고서 쓰는 연습을 했습니다.

자세한 내용 ki0915.github.io/Profile/docs/projects/tryhackme.html

개인 학습, 2023년 개인 프로젝트

WIKI

게시판과 권한별 접근 제한, 실시간 채팅을 갖춘 위키 서비스를 프론트엔드부터 백엔드까지 직접 만든 2023년 개인 프로젝트입니다.

역할
프론트엔드·백엔드 개인 개발

저장소
github.com/ki0915/wiki-backend

저장소 확인 2026-10-16

TypeScript와 Express로 API 서버를, Angular로 클라이언트를 나눠 만들었습니다. 게시글 CRUD와 파일 업로드, JWT 인증과 bcrypt 비밀번호 해시, 역할별 권한을 구현했고, Socket.IO 채팅을 직접 붙였습니다.

자세한 내용 ki0915.github.io/Profile/docs/projects/wiki-backend.html

개인 학습 프로젝트, 2022년

Carrot 클론

상품을 올리고 둘러보는 거래 서비스를 본떠 화면과 서버를 연결해 본 2022년 학습 프로젝트입니다.

역할

React 프론트엔드·Node.js 백엔드 개발

화면은 React와 Axios, Material UI로, 서버는 Express와 MySQL, Sequelize 기반 MVC 구조로 만들었습니다. 두 부분을 연결해 상품 조회, 상세 보기, 등록 기능을 구현했습니다. 채팅 화면은 더미 데이터로 화면 구성만 만들었습니다.

자세한 내용 ki0915.github.io/Profile/docs/projects/carrot.html

학습용 개인 프로젝트, 2021.09–2022.01

Kiosk

2021년 9월부터 2022년 1월까지 주문 키오스크의 상품 선택부터 주문 전송까지 만들어 본 학습용 개인 프로젝트입니다.

역할

개인 개발

C# WPF로 주문 화면을, Node.js와 Express, MSSQL로 서버를 만들었습니다. 상품 목록과 장바구니, 관리자용 상품 관리를 구현하고, 주문 금액은 서버로 보내 검증하게 했습니다. 회사에서 개발한 병원 키오스크 CMS와는 별개의 프로젝트입니다.

자세한 내용 ki0915.github.io/Profile/docs/projects/kiosk.html

경력 EXPERIENCE

미디어솔루션 · 연구개발팀 사원

2023.04–2025.03

미디어솔루션은 직원 50명 이하의 소기업입니다. 병역특례 산업기능요원으로 1년 11개월 근무하며 대형병원, 대형 컨벤션센터, 5성급 호텔 등 4개 기업 고객의 키오스크 CMS와 운영 인프라를 맡았습니다.